Programación Extrema

Proceso de desarrollo de software

- El típico proceso de desarrollo de software consta de las siguientes fases:
- 1. Conceptualización y captura de requisitos.
- 2. Análisis y Descripción funcional
- 3. Diseño
- 4. Codificación
- 5. Pruebas
- 6. Distribución

El problema de la productividad

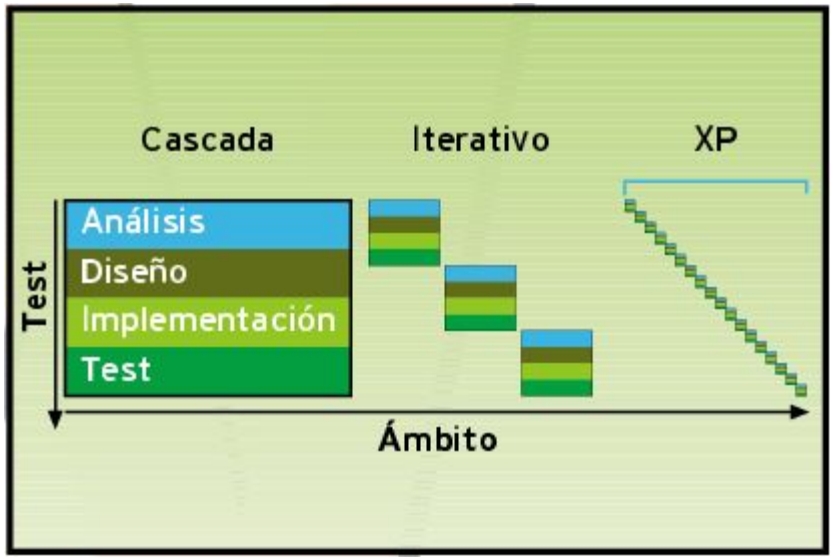
- Los documentos y diagramas se producen de las fases 1 a 3 (desde la Conceptualización hasta el Diseño).
- Estos documentos incluyen la descripción de los requisitos, diagramas UML como casos de uso, diagramas de clases, de actividad, etc.
- Se produce un montón de papeles considerable.
- Este montón de papeles pierde su valor en cuanto se empieza a crear el código, sobre todo si es un sistema que va cambiando con frecuencia dado que no hay tiempo para actualizar toda la documentación y los cambios se hacen sólo en el código.
- Se pierde la conexión entre documentación y código.

¿Por qué fracasan los proyectos de software?

- Retrasos y desviaciones en la planificación.
- Coste de mantenimiento elevados.
- Alta tasa de defectos.
- Requisitos mal comprendidos.
- Cambios de negocio.
- Falsa riqueza de características.
- Cambios de personal.

¿Como soluciona XP estos problemas?

- Retrasos y desviaciones: versiones cortas.
- Cancelan el proyecto: entregas periódicas.
- Sistemas deteriorados y defectos: pruebas continuas.
- Requisitos mal comprendidos: cliente dentro del equipo.
- Cambios de negocio: versiones cortas.
- Falsa riqueza de características: realizar tareas prioritarias.
- Cambios de personal: anima el contacto y la integración.



Metodologías ágiles de desarrollo de software (i)

- Conocidos anteriormente como Metodologías Livianas, los procesos ágiles de desarrollo de software evitan los tortuosos y burocráticos caminos de las metodologías tradicionales y se enfocan en la gente y los resultados.

Metodologías ágiles de desarrollo de software (ii)

- Minimizar la cantidad de esfuerzo y tiempo gastados en construir modelos que sólo servirán como documentación.
- Asegurar que el software entregado funciona para los usuarios.
- Permitir que el proyecto se adapte de manera flexible e inmediata a los cambios originados por tecnologías y/o requisitos.

Metodologías ágiles de desarrollo de software (iii)

- Programación extrema (XP)
- Metodologías Crystal
- SCRUM
- Desarrollo de software adaptativo
- Desarrollo guiado por características (FDD)
- Metodología de desarrollo de sistemas dinámicos (DSDM)

¿Qué es XP?

- “Un proceso ligero, de bajo riesgo, flexible, predecible, científico y divertido de desarrollar software”.

  Kent Beck

  (Extreme Programming Explained)

Características de XP

- Metodología creada a base de prueba y error.
- Surge considerando 4 valores que pueden mejorar cualquier proyecto de software: Simplicidad, Comunicación, Realimentación, Coraje.
- Expresada en forma de 12 prácticas (algunas ya existentes desde hace años), que se soportan las unas a las otras y conforman un conjunto completo.

Los 4 valores (i)

- Simplicidad: XP propone el principio de hacer la cosa más simple que pueda funcionar, en relación al proceso y la codificación. Es mejor hacer algo simple hoy, que hacerlo más complicado hoy y probablemente nunca usarlo.
- Comunicación: Algunos problemas en los proyectos tienen su origen en que alguien no dijo algo a alguien más sobre algo importante en algún momento. XP hace casi imposible la falta de comunicación.

Los 4 valores (ii)

- Realimentación: retroalimentación concreta y frecuente del cliente, del equipo y de los usuarios finales da una mayor oportunidad de dirigir el esfuerzo.
- Coraje: se requiere coraje para confiar en que la retroalimentación durante el camino es mejor que tratar de adivinar todo con anticipación. Se requiere valor para comunicarse con los demás cuando eso podría exponer la propia ignorancia. Se requiere valor para mantener el sistema simple, dejando para mañana las decisiones de mañana. Y, sin un sistema simple, comunicación constante y retroalimentación, es difícil ser valeroso.

XP en la práctica (i)

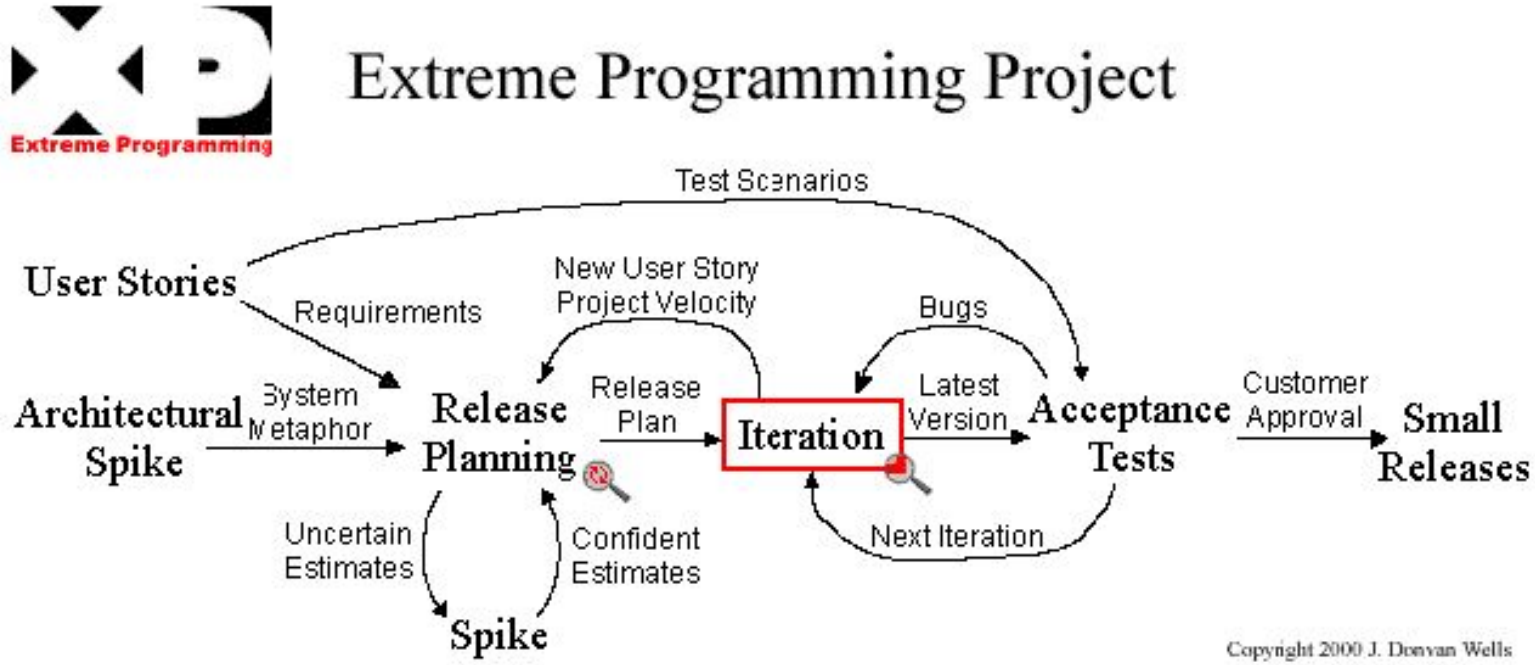
- Retroalimentación a escala fina:
  - ✔ Desarrollo guiado por pruebas
  - ✔ Planificación iterativa
  - ✔ Cliente como parte del equipo
  - ✔ Programación en pares
- Proceso continuo:
  - ✔ Integración continua
  - ✔ Refactorización
  - ✔ Liberación pequeña, entregas frecuentes

XP en la práctica (ii)

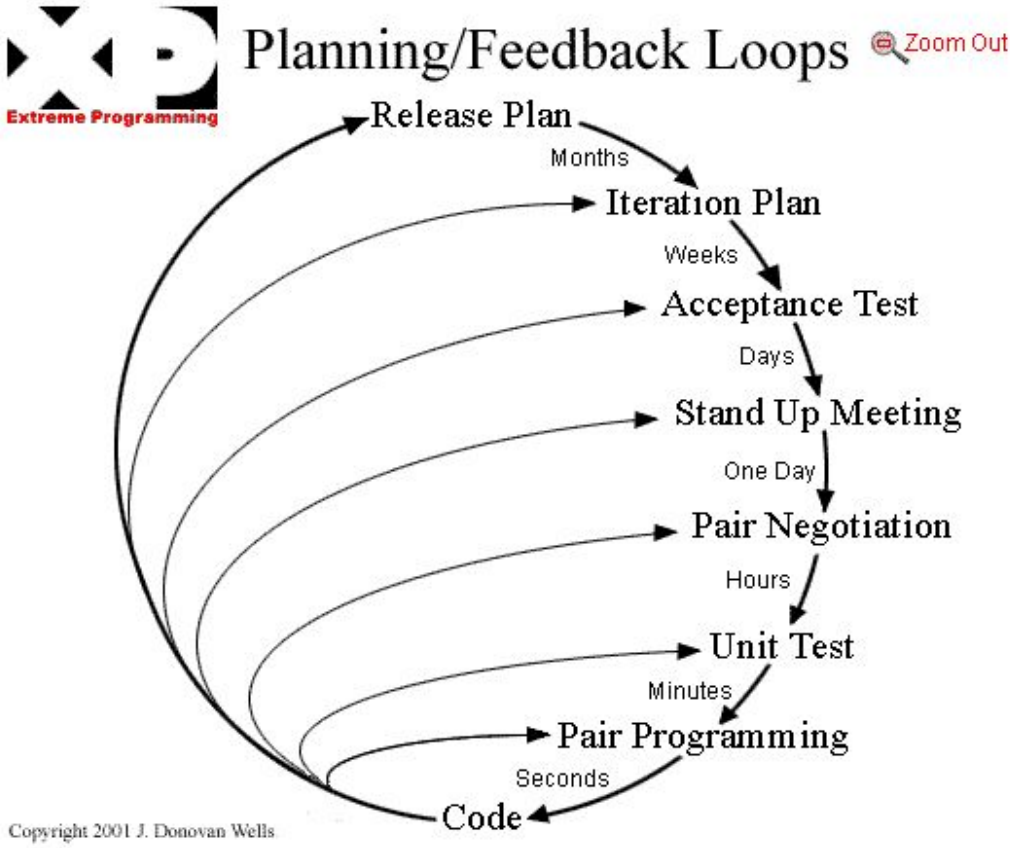
- Entendimiento compartido:
  - ✔ Diseño simple
  - ✔ Metáforas del sistema
  - ✔ Propiedad colectiva del código
  - ✔ Estándares de codificación

- Bienestar del programador:
  - ✔ Ritmo sostenible (Semanas de 40 horas)

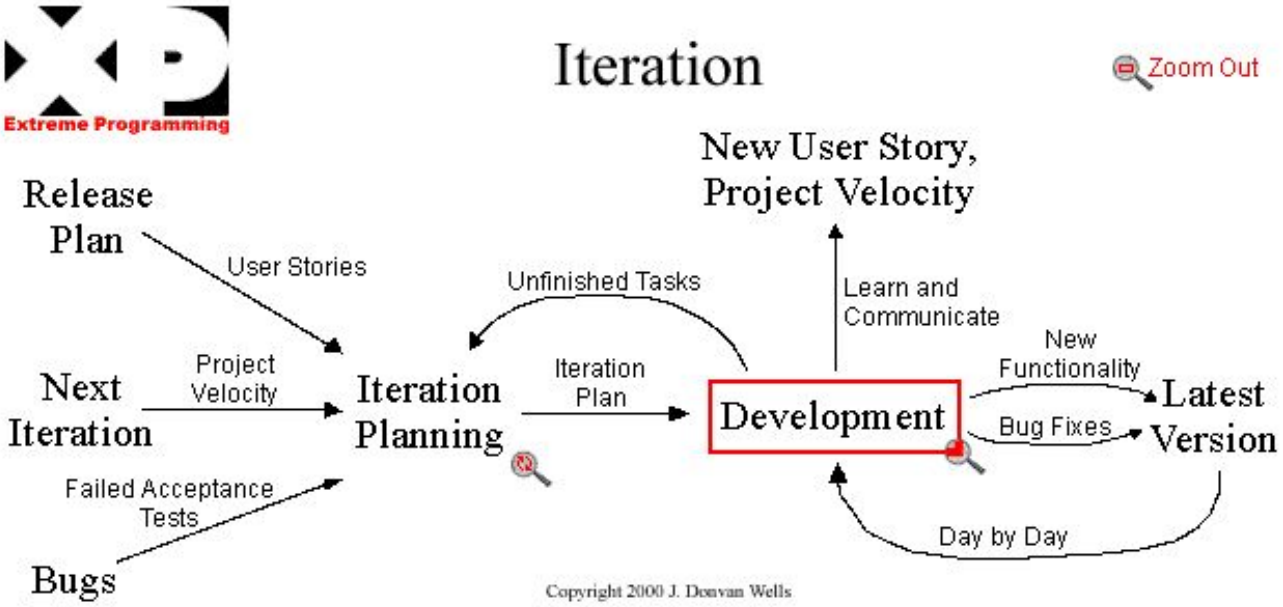
Proceso de desarrollo de software con XP (i)



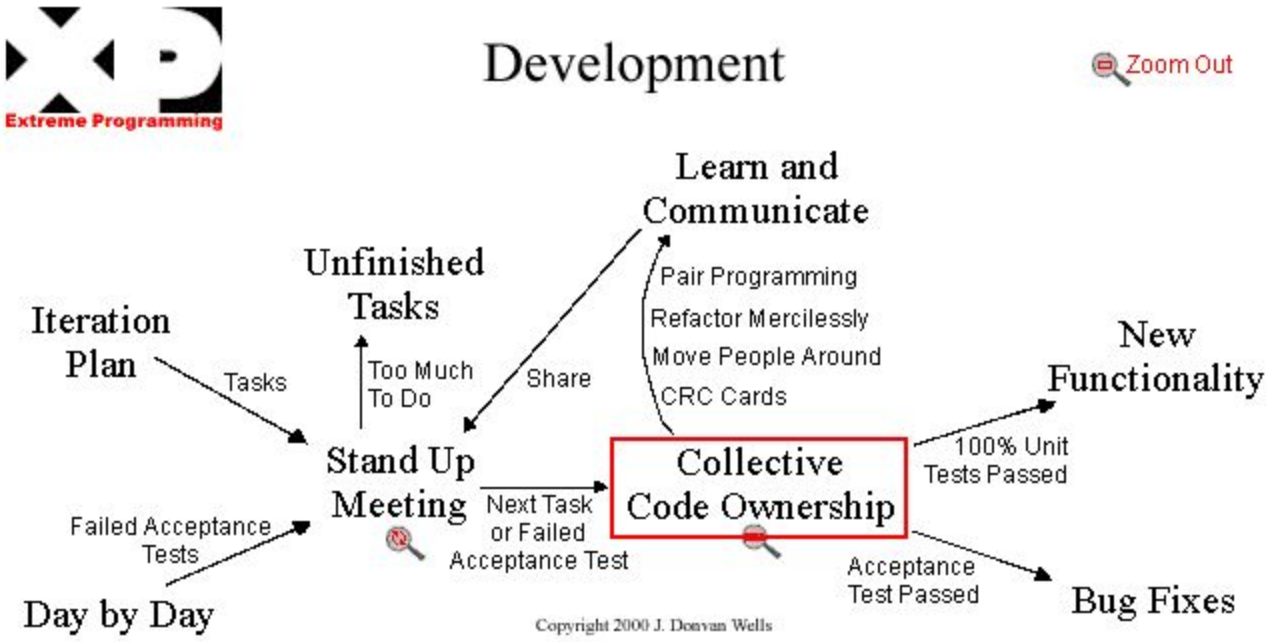
Proceso de desarrollo de software con XP (ii)



Proceso de desarrollo de software con XP (iii)



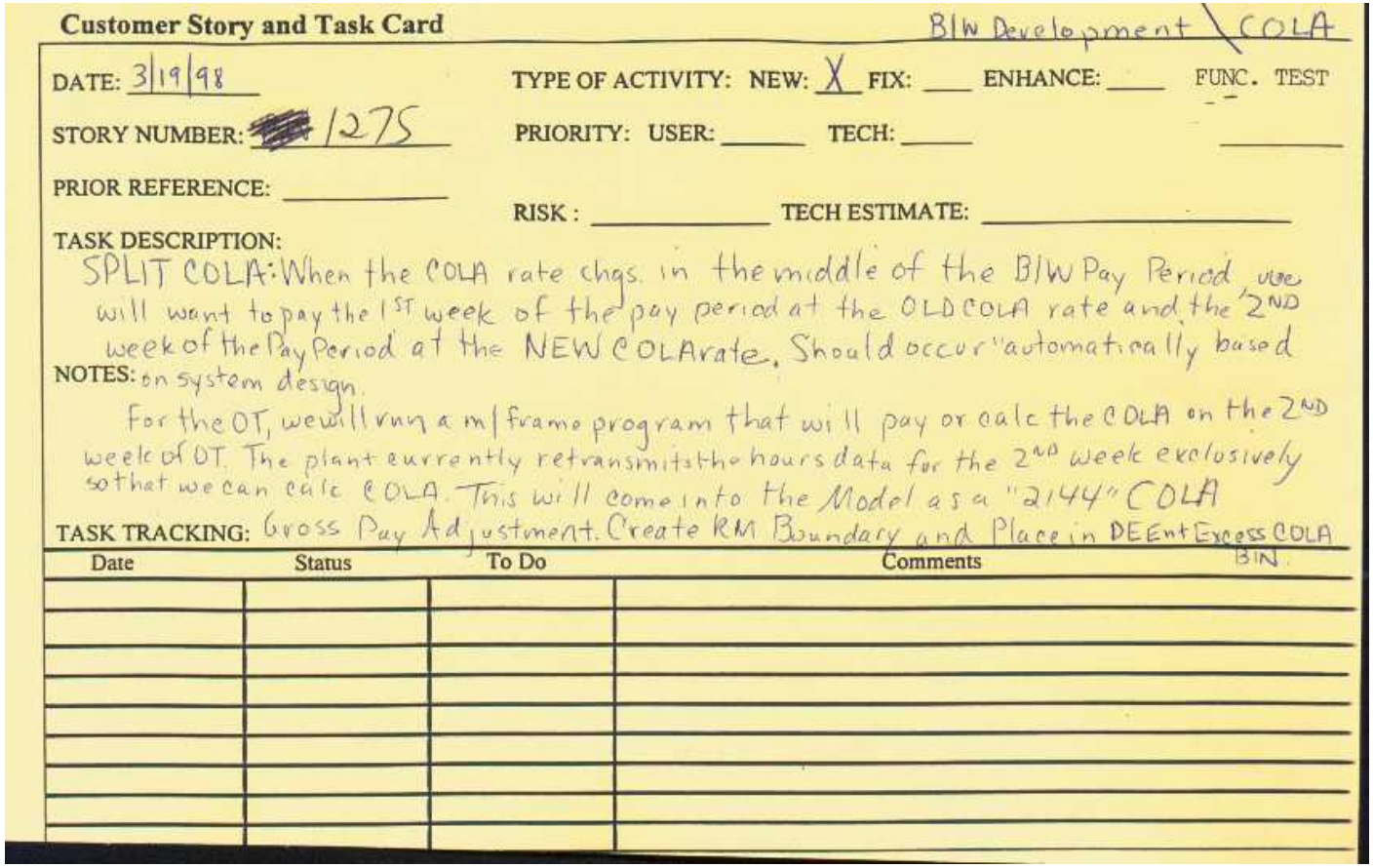
Proceso de desarrollo de software con XP (iv)



Relatos (historias) de Usuario (i)

- Una Historia de usuario es un relato acerca de qué problema debe resolver el sistema. Cada relato se escribe en una tarjeta y representa una parte de la funcionalidad que es coherente para el cliente.
- Son escritos por el cliente o usuario, con la ayuda de los desarrolladores, para permitir estimar los tiempos y asignar prioridades.
- Los clientes ayudan a asegurar que la mayoría de la funcionalidad deseada para el sistema está cubierta con las historias.
- Constan de 3 ó 4 líneas escritas por el cliente en un lenguaje no técnico sin hacer mucho hincapié en los detalles; no se debe hablar ni de posibles algoritmos para su implementación, ni de diseños de base de datos adecuados, etc.

Relatos (historias) de Usuario (ii)



Customer Story and Task Card — B/W Development \ COLA

DATE: 3/19/98 — TYPE OF ACTIVITY: NEW: X FIX: ___ ENHANCE: ___ FUNC. TEST ___

STORY NUMBER: [redacted] 1275 — PRIORITY: USER: ___ TECH: ___

PRIOR REFERENCE: ___ — RISK: ___ TECH ESTIMATE: ___

TASK DESCRIPTION:

SPLIT COLA: When the COLA rate chgs. in the middle of the B/W Pay Period, user will want to pay the 1ST week of the pay period at the OLD COLA rate and the 2ND week of the Pay Period at the NEW COLA rate. Should occur automatically based on system design.

NOTES:

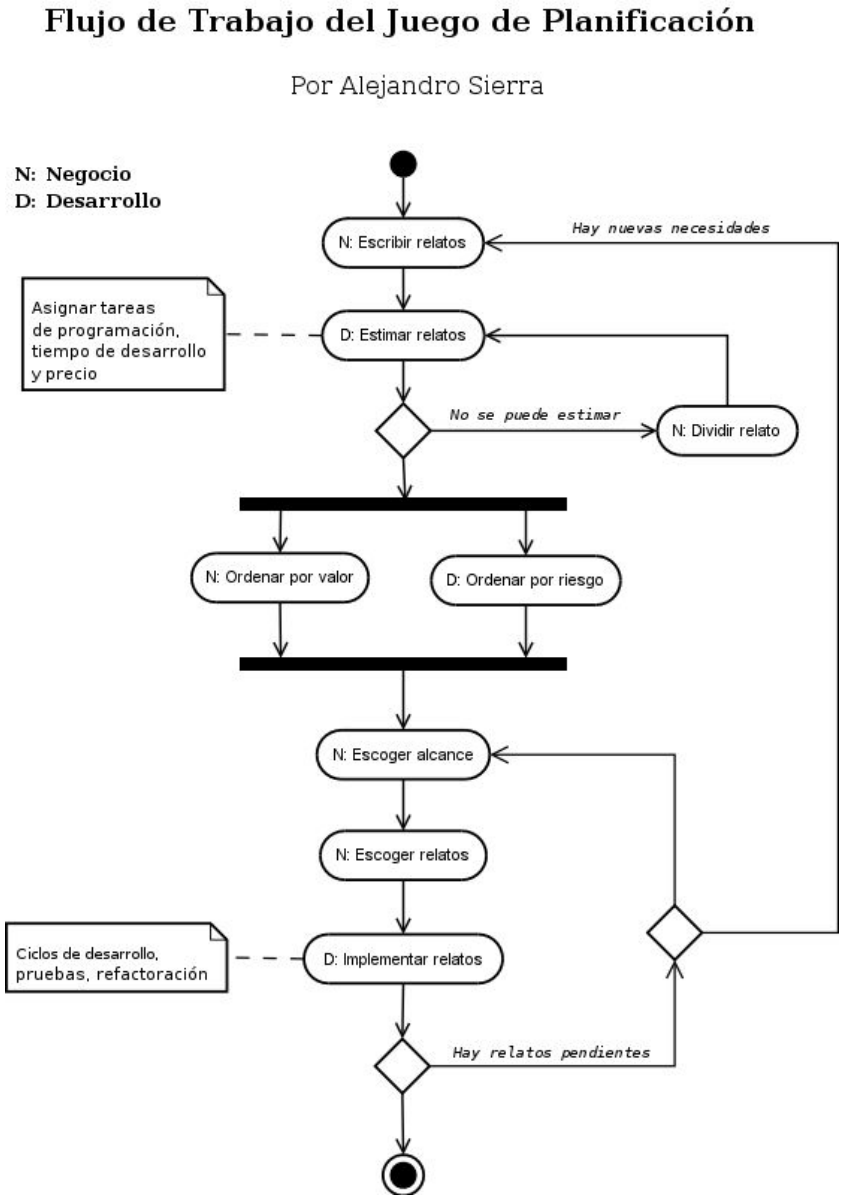
For the OT, we will run a m/frame program that will pay or calc the COLA on the 2ND week of OT. The plant currently retransmits the hours data for the 2ND week exclusively so that we can calc COLA. This will come into the Model as a "2144" COLA

TASK TRACKING: Gross Pay Adjustment. Create RM Boundary and Place in DEEntExcess COLA BIN.

Date	Status	To Do	Comments

Planificación

- En el juego de planificación, el cliente y los programadores negocian el alcance del proyecto para cada iteración.
- El factor crítico es permitir al cliente tomar las decisiones de negocio y al equipo de desarrollo tomar las decisiones técnicas.



Diseño simple

- El diseño debe ser lo más simple posible: no introducir estructura, ni funcionalidad antes de tiempo.
- Se puede añadir complejidad más adelante.
- Inconveniente: Vencer la tendencia al “gran diseño previo”

Pruebas automatizadas (i)

- Todo código que puede fallar debe tener una prueba.
- Hacer la prueba aún antes de la implementación.
- Inconveniente: Obliga a imponer una forma de trabajar y puede ser necesaria formación/experiencia.
- Dos tipos: Prueba de Unidad (o del Programador) y Prueba de Aceptación (o Funcional, o del Cliente).
- La Prueba de Aceptación es una prueba formal conducida para determinar si un sistema satisface los criterios de aceptación y permite al cliente determinar si acepta o no el sistema.

Pruebas automatizadas (ii)

- Para cada lenguaje de programación hay herramientas de Prueba de Unidad que permiten automatizar la ejecución de las mismas, como JUnit para Java. (ver http://www.xprogramming.com/software.htm)
- Frecuentemente una Prueba de Unidad es mejor que un comentario para ayudar a entender por qué una determinada función es necesaria, para demostrar cómo es llamada una función y cuales son los resultados esperados, y para documentar defectos en versiones previas del programa que queremos asegurarnos de que no vuelvan.

Integración continua

- Todos los cambios deben ser integrados a la base del código al menos diariamente.
- Las pruebas deben correr al 100% antes y después de la integración.
- Cada nueva versión debe tener la mínima funcionalidad extra que tiene sentido.
- Ventajas: tener realimentación de los usuarios y ofrecer pronto nueva funcionalidad (+éxito).

Programación en pares

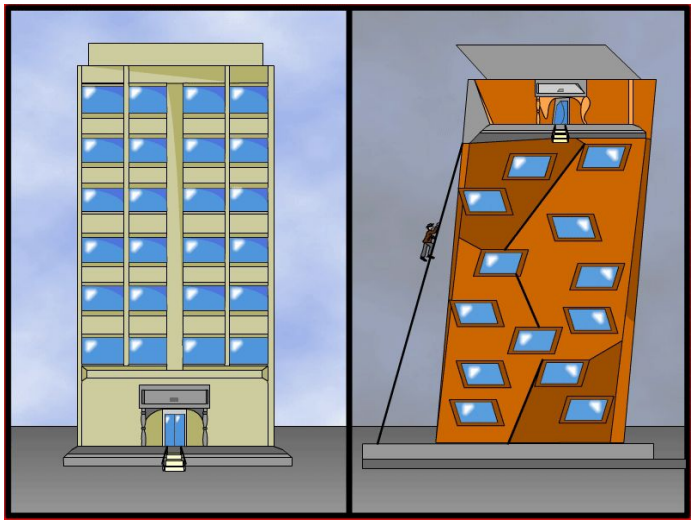
- La Programación en Pares requiere que dos desarrolladores participen en un proyecto en una misma estación de trabajo.
- Cada miembro lleva a cabo la acción que el otro no está haciendo en ese momento: Mientras uno redacta Pruebas de Unidad el otro piensa acerca de la clase que satisfará a dicha prueba, por ejemplo.
- Los estudios demuestran que, tras aprender Habilidades Personales dos programadores son más que doblemente productivos que uno sólo para una tarea determinada.



Refactorización (i)

- Es una técnica disciplinada de reestructurar cualquier código existente, alterando su estructura interna sin modificar su comportamiento externo.

¿Si su software fuera un edificio, se parecería mas a uno de la izquierda o de la derecha?



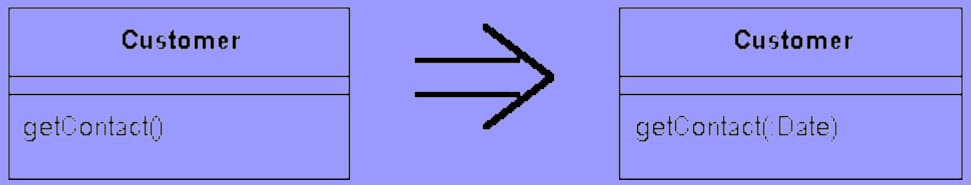
Refactorización (ii)

- Si un programa funciona pero está mal diseñado, pronto surgirán problemas a la hora de actualizarlo. Los problemas más comunes pueden ser catalogados por el código".
- Existen listas de refactorizaciones. Ejemplo:

Add Parameter

A method needs more information from its caller.

Add a parameter for an object that can pass on this information.

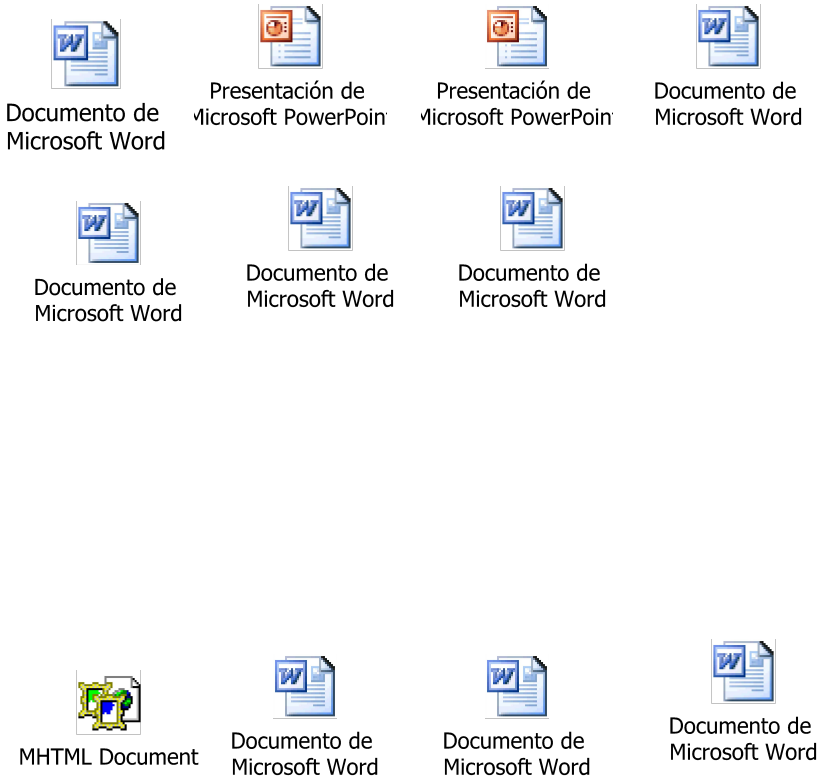


Caso de Estudio (i)

- Tomado de Universidad Politécnica de Valencia (España) http://www.dsic.upv.es/asignaturas/facultad/lsi/ejemploxp/
- El proyecto consiste en el desarrollo de un sistema de gestión para una empresa de confecciones. En dicha gestión de la empresa se incluyen gestión de pedidos, gestión de clientes (tanto principal como los de temporada), facturación, gestión de productos, gestión de materias primas, etc...

Caso de Estudio (ii)

- Gestión del proyecto:
  - Planificación del proyecto
  - Diario de Actividades
- Implementación:
  - Base de Datos
  - Interfaces de Usuario
  - Código Fuente
- Pruebas



sistema de gestión para una empresa de confecciones.

Sistema de Gestión para Empresa de Confecciones

Módulos principales:

1. Gestión de clientes
2. Gestión de productos
3. Gestión de materias primas
4. Gestión de pedidos
5. Facturación
6. Control básico de inventario
7. Reportes

Y el desarrollo puede organizarse así:

Planificación XP, Historias de usuario, Iteraciones, Diseño simple, BD, Interfaz, Código, Pruebas, Retroalimentación, Nueva iteración

sistema de gestión para una empresa de confecciones.

Gestión del proyecto con XP
Para el caso sugiero que el proyecto tenga **4 iteraciones**.

Iteración	**Funcionalidad**	**Producto entregable**
Iteración 1	Clientes + Productos	CRUD funcionando
Iteración 2	Materias primas + Inventario	CRUD + movimientos
Iteración 3	Pedidos	Registro y seguimiento de pedidos
Iteración 4	Facturación + Reportes	Sistema integrado

Planificación del proyecto

Prácticas fundamentales de XP:

Historias de usuario
Por ejemplo:
HU01 – Registrar cliente
Como usuario del sistema, quiero registrar los datos de un cliente para poder gestionar sus pedidos.
Criterios de aceptación:

- Registrar código.
- Registrar razón social o nombres.
- Registrar RUC/DNI.
- Registrar teléfono.
- Registrar correo.
- Registrar dirección.
- Indicar si es cliente principal o de temporada.
- Validar datos obligatorios.
- Guardar información en la base de datos.

Planificación del proyecto

HU02 – Registrar producto
Como usuario del sistema, quiero registrar los productos confeccionados para poder gestionar los pedidos de los clientes.
Datos:

Código
Nombre
Descripción
Talla
Color
Precio
Stock
Estado

Planificación del proyecto

HU03 – Registrar materia prima
Como encargado de almacén, quiero registrar las materias primas para controlar los recursos utilizados en la confección.

Ejemplos:

Tela
Hilo
Botones
Cierres
Etiquetas

Planificación del proyecto

HU04 – Registrar pedido
Como vendedor, quiero registrar los pedidos de los clientes para controlar los productos solicitados y su estado.
El pedido podría tener:
Número de pedido
Cliente
Fecha
Producto
Cantidad
Precio
Importe
Estado
Estados:
Pendiente En producción Terminado Entregado

Priorización XP

Priorizar las historias.

Esto permite explicar que **XP trabaja con entregas pequeñas y frecuentes**, no con un proyecto que se desarrolla completamente y recién se prueba al final.

ID	Historia	Prioridad	Iteración
HU01	Registrar clientes	Alta	1
HU02	Registrar productos	Alta	1
HU03	Registrar materias primas	Alta	2
HU04	Controlar inventario	Alta	2
HU05	Registrar pedidos	Muy alta	3
HU06	Actualizar estado pedido	Alta	3
HU07	Generar factura	Alta	4
HU08	Consultar reportes	Media	4

Programación o diario de Actividades

El diario sea una herramienta obligatoria.
Esto es importante porque **no debe ser simplemente un diario narrativo**.
Debe permitir responder:
¿Qué hicimos?
¿Qué problema encontramos?
¿Cómo lo solucionamos?
¿Qué quedó pendiente?

Fecha	Actividad	Responsable	Resultado	Problema	Solución
07/10	Crear BD	Ana	BD creada	Error conexión	Configuración servidor
08/10	Crear tabla Cliente	Luis	Tabla creada	Campo RUC	Se agregó validación
09/10	Formulario Cliente	Pedro	Interfaz terminada	Error guardar	Se corrigió conexión
10/10	Prueba CRUD	Equipo	CRUD funcionando	Error eliminar	Se corrigió consulta